SOFTWARE REQUIREMENTS SPECIFICATION

for

INVESTCIRCLE

Version 1.0

Prepared by : 1. Ishan Jha (1024160099)
2. Lakshay Sachdeva (1024160104)
3. Aarav K. Arora (1024160106)
4. Anirudh Phophalia (1024160120)

Submitted to : Anushka Ma'am
Thapar Institute of Engineering and Technology (UCS503)

October 15, 2026

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Intended Audience and Reading Suggestions . . . 4
- 1.3 Project Scope . . . 6

2 Overall Description **8**
- 2.1 Product Perspective . . . 8
- 2.2 User Classes and Characteristics . . . 10
- 2.3 Product Functions . . . 12
- 2.4 Operating Environment . . . 14
- 2.5 Design . . . 18

3 System Features **25**
- 3.1 Description and Priority . . . 25
- 3.2 Functional Requirements . . . 28

4 Other Nonfunctional Requirements **38**
- 4.1 Performance Requirements . . . 38
- 4.2 Security Requirements . . . 40
- 4.3 Software Quality Attributes . . . 42
- 4.4 Business Rules . . . 46

5 Other Requirements **47**

1 Introduction

This chapter gives a scope description and overview of everything included in this SRS document. It also describes the purpose of the document, lists the definitions and abbreviations used, and explains how the remaining chapters are organised.

1.1 Purpose

Retail investors research stocks across social media, discussion forums, financial news websites and informal stock-tip communities. On these platforms predictions are rarely recorded in a structured form, their outcomes are not tracked, and credibility is judged by followers or likes rather than by historical accuracy. So, "InvestCircle" is the solution. "InvestCircle" is a web-based, peer-to-peer financial research platform that combines identity-based stock discussion, structured stock calls that are automatically evaluated against real market data, and a personalised scrollable market-news feed. The main concept of "InvestCircle" is to build a persistent, public track record for every structured call, so that credibility is earned rather than claimed.
The purpose of this document is to give a detailed description of the requirements for "InvestCircle" Version 1.0. It illustrates the purpose and complete declaration for the development of the system, and explains its constraints, interfaces and interactions with external services such as the market-data provider. This document is intended to be proposed to the course instructor for approval and to serve as a reference for the development team while building the first version of the system. It is derived from, and consistent with, the approved project proposal (UCS503).
Overview of the document. Chapter 1 introduces the project, its scope, objectives and stakeholders. Chapter 2 gives an overview of the system, its users, constraints and assumptions, and its interfaces and design. Chapter 3 lists the features, explains how they are prioritised and released, and specifies the functional requirements in detail. Chapter 4 specifies the quality requirements (performance, security, quality attributes) and business rules. Chapter 5 contains other requirements, the risk register, open issues and references.

1.2 Intended Audience and Reading Suggestions

This SRS is for developers, project managers, users, testers and the course instructor. Further the discussion will provide all the internal, external, functional and also non-functional informations about "InvestCircle". Developers should read all chapters; project managers Chapters 1, 2 and 5 and Section 3.1; testers Chapters 3 and 4, since every requirement is written to be verifiable; the instructor may read Chapter 1 and then sample the requirements in Chapter 3.
Requirements are identified as **FR** (functional requirement) and **QR** (quality requirement). Functional requirements use the fields ID, TITLE, DESC (description), RAT (rationale) and DEP (dependency). Quality requirements use the fields ID, TAG, GIST, SCALE, METER, MUST, PLAN and WISH. Priorities follow MoSCoW: **Must** (required for the MVP), **Should** (subsequent iterations) and **Could** (optional extension).
Definitions, acronyms and abbreviations.

Term	Definition
User	Anyone who interacts with the InvestCircle website (guest, registered user or contributor)
Guest	A visitor who is not registered or logged in
Registered user	A user with an account who can post, vote, follow and manage watchlists
Contributor	A registered user who has published at least one structured call
Moderator / Admin	A user with special permission to review reports, remove content and manage accounts
Post	A research note, question or opinion published in the Discussions tab
Call (stock call)	A structured prediction made of ticker, direction (bullish/bearish), target price and time horizon, attached to a post
Explicit call	A call entered by the user through the structured form
AI-inferred call	A call detected by an AI component from free text; always labelled "AI-inferred"
Pending / Hit / Miss	Status of a call: not yet resolved / target reached within the horizon / target not reached
Horizon	The deadline by which the target price should be reached
Track record	The public history and accuracy score of a user's confirmed calls
Ticker	Short symbol identifying a listed security
Watchlist	A user-maintained list of tickers
News Reel	The vertically scrollable, reels-style market-news feed
Personal page	A user's profile showing posts, followers and track record
Provider layer	The service layer that hides the external market-data API from the rest of the system
MVP	Minimum viable product (initial deliverable)
API / REST	Application Programming Interface / Representational State Transfer
JWT	JSON Web Token, used for token-based authentication
OAuth	Open standard for delegated login
ORM	Object-Relational Mapper
CI/CD	Continuous Integration / Continuous Delivery
DESC, RAT, DEP	Description, Rationale, Dependency
TAG, GIST	A unique identifier / a short description of a quality requirement
SCALE, METER	The scale of measure / the process used to measure it
MUST, PLAN, WISH	Minimum acceptable level / level at which success is claimed / desirable level

Table 1.1: Definitions, acronyms and abbreviations

1.3 Project Scope

"InvestCircle" creates a space for Readers, Contributors and Moderators to do financial research and discussion under real identities. It is a web-based application with a responsive interface, used from any modern browser. It helps people to publish stock opinions in a structured way, to follow contributors whose past calls proved accurate, and to read market news that is relevant to the stocks they care about. The application is free to use and needs an Internet connection.
Registered users publish research posts in the Discussions tab. A post can optionally carry a structured call. The system stores the call as Pending and, when the horizon expires, a background process obtains market-price data and marks the call as Hit or Miss. The result becomes part of the author's public track record, shown on the Personal page. Users also maintain watchlists of tickers, which personalise the News Reel. Moderators use privileged functions of the same website to keep the platform clean and the information accurate.
It is for **research and discussion, not trading**: it will not execute trades, hold funds or connect to any brokerage account.
Objectives.

1. Accountability: keep a permanent public record of every structured call and its outcome, tied to the author's real identity.

2. Measurable credibility: use historical accuracy instead of likes or follower counts as the main credibility signal.

3. Consolidated research: bring discussion, news and watchlists into one platform.

4. Useful scrolling: offer a fast scrollable news feed that replaces unproductive short-form scrolling.

5. Reliability: resolve at least 95% of due calls automatically.

6. Engineering quality: a modular, secure, scalable system with at least 70% automated test coverage of backend and business logic.

In scope. User registration, authentication, profiles and follow/unfollow; a Discussions tab with posts, voting, comments and threaded replies; structured stock calls with automatic resolution and a public accuracy record; watchlists; a News Reel personalised by watchlist; notifications; search and discovery; contributor statistics; basic moderation.
Optional extensions (outside the core MVP). Sector/sentiment tagging, AI-inferred call detection (always labelled "AI-inferred"), short video reactions to news ("Share Your Opinion"), advanced recommendations, analytics and rankings.
Out of scope. Trade execution, brokerage integration, payment processing, investment advice, novel price-prediction algorithms and native mobile apps (a responsive web interface is provided).
Stakeholders.

Stakeholder	Role	Main interest
Readers (retail investors)	Browse discussions, follow users, use News Reel and watchlists	Trustworthy research and relevant news
Contributors	Publish posts and structured calls	Earn credibility through a verifiable record
Moderators / Admins	Review reports, remove content, manage accounts	Platform quality and safety
Development team	Design, build, test, deploy	Clear, testable requirements
Course instructor	Reviews and evaluates the project	Consistency with the approved proposal
Market-data provider	Supplies prices and news (Alpha Vantage, Finnhub or similar)	Fair, rate-limited API use

Table 1.2: Stakeholders of InvestCircle

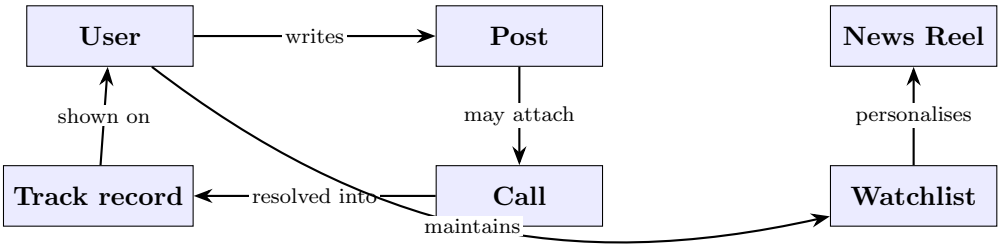


Figure 1.1: Entire work-flow

Figure 1.1 (Entire work-flow) is the overview of the project. A "User" writes a "Post"; a "Post" may attach a "Call"; every "Call" is resolved into the user's "Track record", which is shown on the user's profile. A user's "Watchlist" personalises the "News Reel". So, every entity is very interactive with each other.

2 Overall Description

This chapter gives an overview of the whole system. The system is explained in its context to show how it interacts with other systems, and its basic functionality is introduced. It also describes the types of users, the interfaces, the constraints and assumptions, and the design of the main workflows.

2.1 Product Perspective

"InvestCircle" is a new, self-contained web application. It replaces the scattered, untracked way retail investors currently share stock opinions across social media and forums, and depends on external services only for market data. Main goal of this project is to make financial discussion accountable, transparent and measurable.
The system consists of a web client and a server side. The web client is used to read and write posts, calls and watchlists and to scroll the News Reel; the server side stores all data, resolves calls in the background and talks to the market-data provider. It follows a layered client–server architecture (Figure 2.1):

- **Frontend:** React single-page application, responsive, with light and dark mode.
- **Backend API:** Node.js REST services for authentication, users, posts, comments, calls, watchlists and notifications.
- **Business logic:** call resolution, feed ranking, follower relationships, notification generation.
- **Database:** PostgreSQL for persistent relational data. **Cache/queue:** Redis for caching market data and feeds and for background-job support.
- **External services:** financial-market APIs and optional AI services, isolated behind a provider-abstraction layer so that providers can be replaced.

The web client communicates only with the REST API over HTTPS. The API reads and writes the database and never lets the client talk to the market-data provider directly. Background workers perform operations that must not block user requests: market-data synchronisation, expired-call resolution, notification generation and news-feed updates.

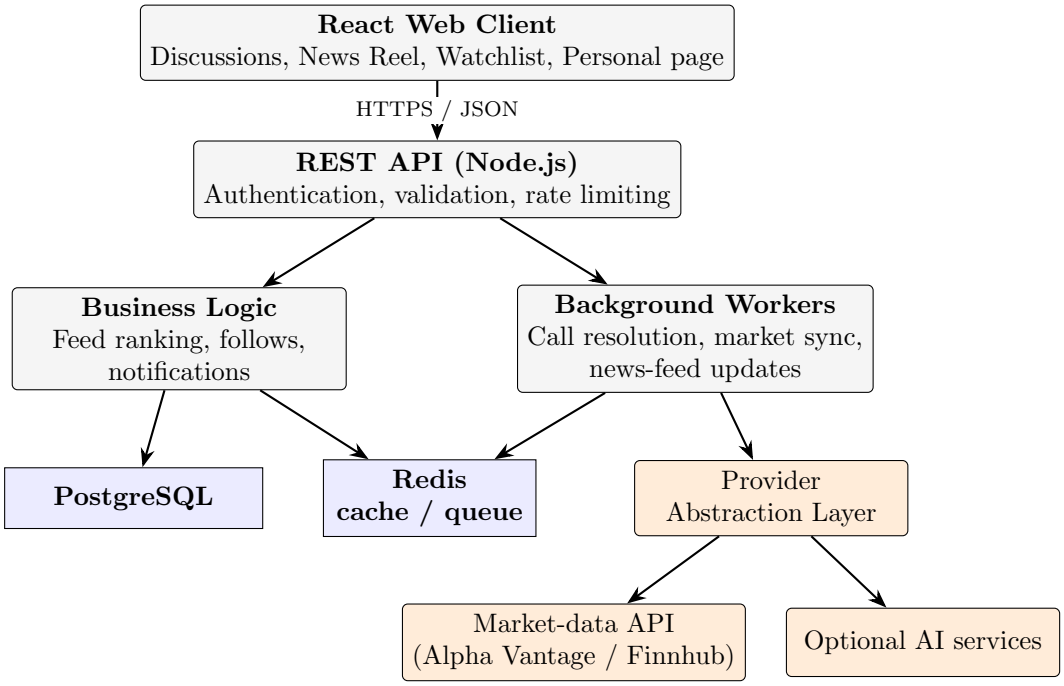


Figure 2.1: High-level system architecture

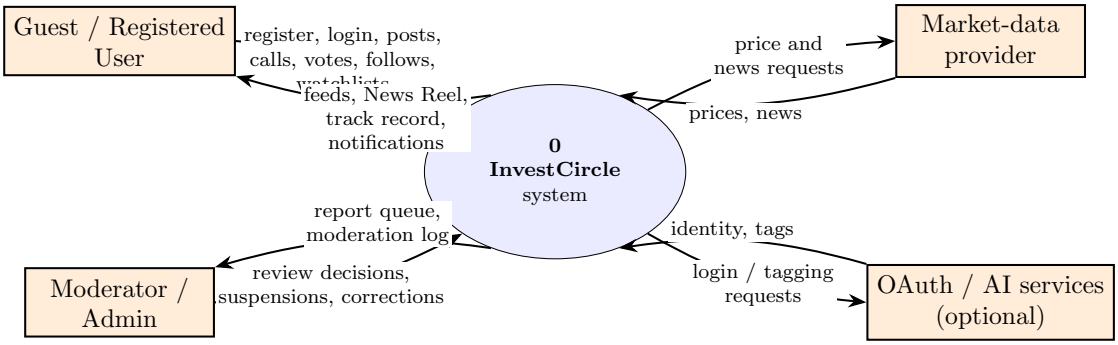


Figure 2.2: DFD Level 0 - context diagram of InvestCircle

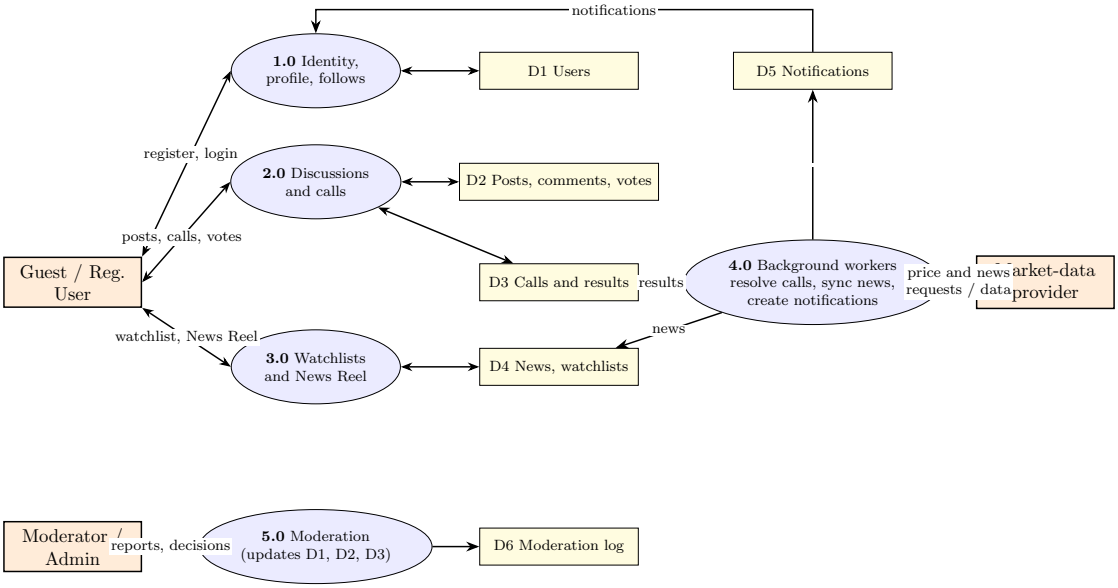


Figure 2.3: DFD Level 1 - main processes and data stores

In the DFDs, ellipses are processes, thick rectangles are external entities and the plain rectangles D1–D6 are data stores.

2.2 User Classes and Characteristics

"InvestCircle" has basically 4 types of users.

- Guest (unregistered)
- Registered User
  - Reader
  - Contributor (a user who has published structured calls)
- Moderator / Admin
- External systems (market-data provider, OAuth provider, optional AI service)

Each of these types of users has different use of the system, so each of them has their own requirements.
Guest. A guest can only view limited public content (landing page and public posts) and cannot post, vote, follow or comment. A guest has to register before using the main functions.
Registered User (Reader / Contributor). A registered user posts, votes, comments, follows other users, manages watchlists, uses the News Reel and owns a Personal page. A

Reader mainly consumes content; a Contributor additionally publishes structured calls and is therefore judged by the accuracy of those calls. Both need only basic web-browsing skill and use the system through a browser on a desktop or mobile screen.

Moderator / Admin. A moderator reviews reported content, removes violating posts and comments, suspends accounts and, when necessary, corrects a resolved call (every action is logged). Moderators need medium technical skill and use the privileged part of the same website.

External systems. The market-data provider supplies prices and news; the OAuth provider (optional) supplies federated login; the AI service (optional) supplies tagging and inferred-call detection. External systems never use the user interface.

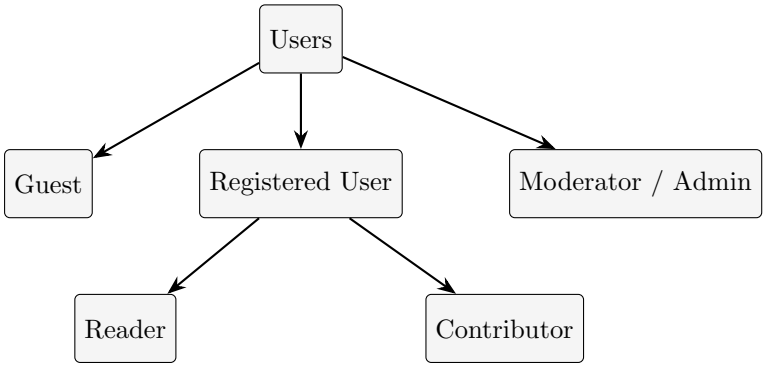


Figure 2.4: type of users

Capability	**Guest**	**Reader**	**Contributor**	**Moderator / Admin**
View limited public content	Yes	Yes	Yes	Yes
Register / log in	Yes	–	–	–
Post, vote, comment, reply	–	Yes	Yes	Yes
Follow users, receive notifications	–	Yes	Yes	Yes
Watchlists and News Reel	–	Yes	Yes	Yes
Publish structured calls	–	–	Yes	Yes
Has a public track record	–	–	Yes	Yes
Review reports, remove content	–	–	–	Yes
Suspend accounts, correct calls	–	–	–	Yes

Table 2.1: Capabilities of each user class

2.3 Product Functions

"InvestCircle" stores all posts, calls and call outcomes of its users and maintains their public track records. With the website, users will be able to register and keep a persistent profile; publish research posts with an optional structured call; vote, comment and reply in threads; follow other users and receive notifications; maintain watchlists; scroll a personalised News Reel; search users, stocks and discussions; and have every call automatically resolved as Hit or Miss.
The Discussions feed shows posts in a paginated list with the author's identity, ticker, votes and the call badge. The News Reel shows one news item at a time in a vertically scrolling feed, ranked so that items matching the user's watchlist appear more prominently. The Personal page shows the user's own posts, followers, watchlist and the history and accuracy of resolved calls. The administrator side provides moderation of reported content and of accounts. The system also gives each user notifications about new followers, comments and resolved calls.
Before using the main functions, users have to be registered. The principal data entities (Figure 2.5) are:

- **users** - user_id, full_name, email (unique), password_hash, oauth_provider, bio, role, verified, status, created_at.
- **posts** - post_id, author_id (foreign key), title, body, ticker, upvotes, downvotes, created_at, updated_at, is_deleted.
- **calls** - call_id, post_id (foreign key, unique), author_id, ticker, direction, target_price, reference_price, horizon_date, status (Pending/Hit/Miss), is_ai_inferred, resolved_at, resolution_price, data_source. A post can optionally reference one call; the flag separates explicit from AI-inferred calls.
- **comments** - comment_id, post_id, author_id, parent_comment_id (self-reference for threaded replies), body, created_at.
- **votes** - vote_id, post_id, user_id, value (+1/-1); one vote per user per post.
- **follows** - follower_id, followee_id, created_at.
- **watchlists** and **watchlist_items** - owner_id, name, ticker, added_at.
- **notifications** - recipient_id, type (follow/comment/call_resolved), reference_id, is_read, created_at.
- **news_items** - headline, source, published_at, ticker, category, url. **reactions** - news_id, user_id, video_ref (optional feature).

Foreign keys maintain referential integrity; indexes are applied to user IDs, tickers, call status/horizon and timestamps. Call is the main feature of all, so resolved calls are retained permanently and are immutable except for logged administrator corrections.

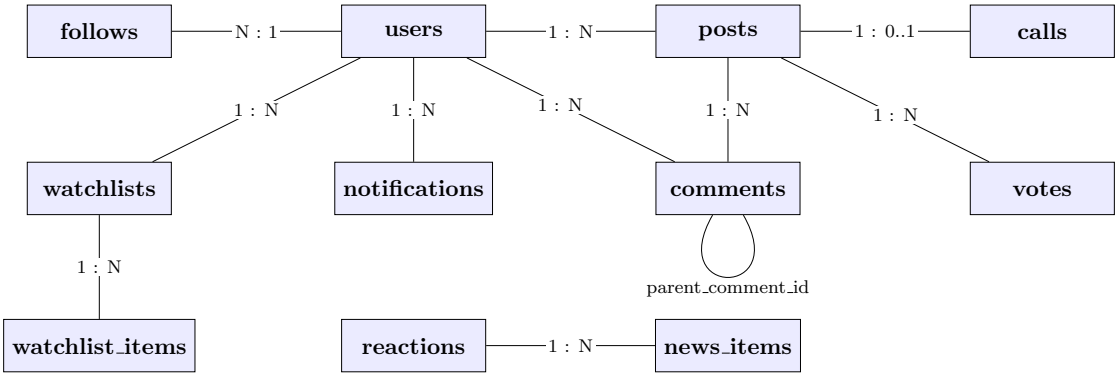


Figure 2.5: Data Flow Diagram (entity relationships)

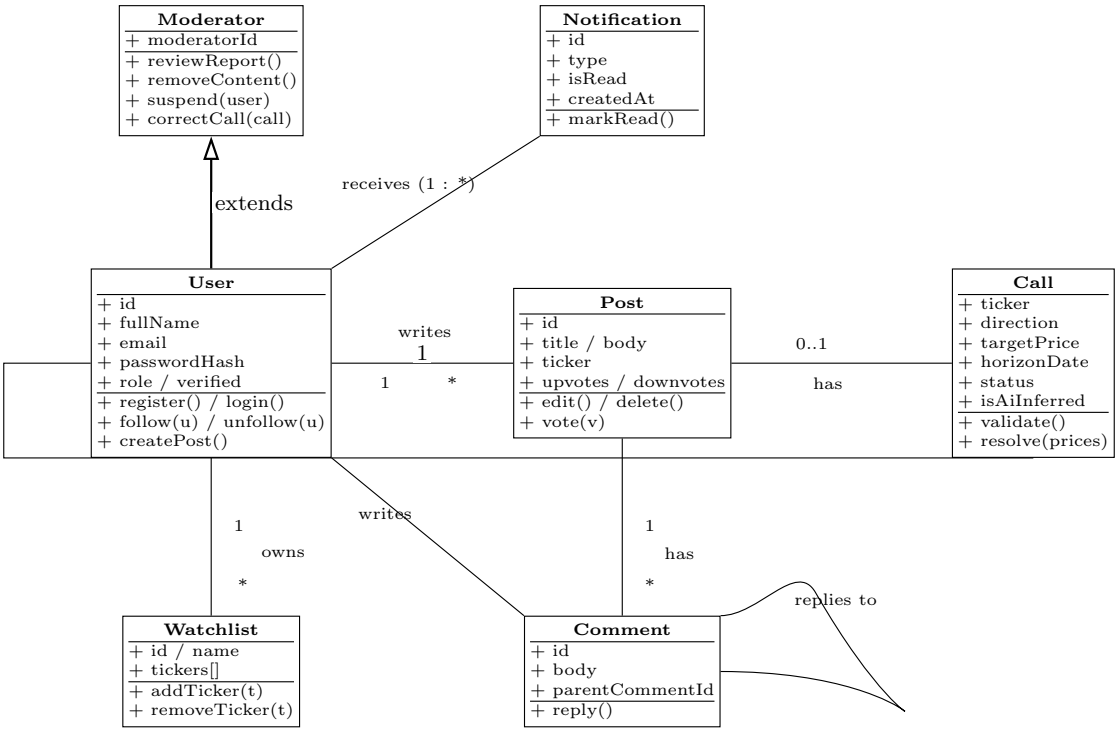


Figure 2.6: Class diagram - domain classes

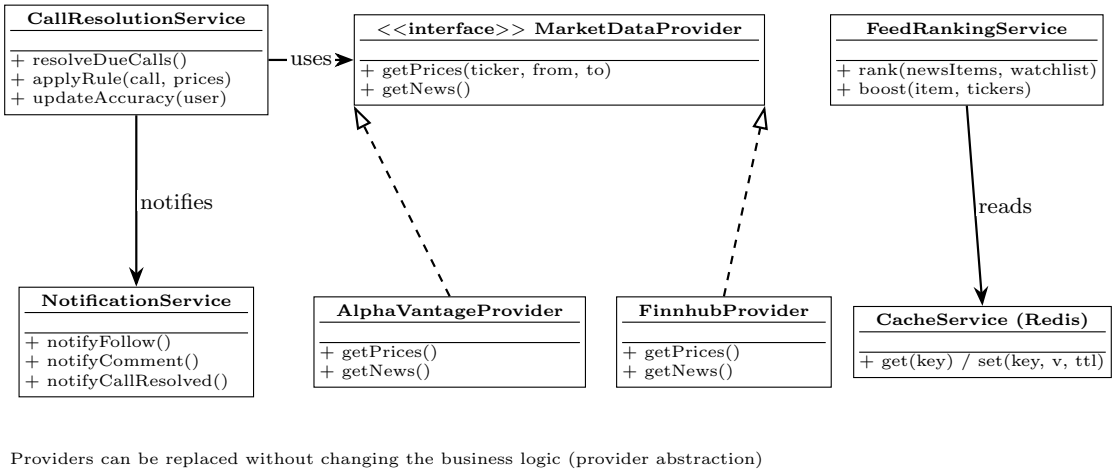


Figure 2.7: Class diagram - service and provider classes

2.4 Operating Environment

The website will be operate in any modern web browser (current Chrome, Firefox, Edge, Safari) on Windows, Mac, Linux, Android or iOS with a responsive layout. The server side runs as containerised Node.js services on common cloud hosting with managed PostgreSQL and Redis. An Internet connection and HTTPS are required.

External interface requirements

User interfaces. A first-time visitor sees the landing page with the login and registration form (Figure 2.8, left). A logged-in user lands on the Discussions tab, which is the default section (Figure 2.8, middle). Here the user can sort posts, filter by ticker and open a post. A user who wants to publish a call opens the post composer, writes the research text, enables "Attach call" and enters ticker, direction, target price and horizon (Figure 2.8, right). Inline validation messages appear next to invalid fields.
Every user has a Personal page (Figure 2.9, left) showing biography, followers/following, accuracy score, tabs for Posts, Calls and Watchlist, and the history of resolved calls with a Hit/Miss badge. The News Reel (Figure 2.9, middle) shows one news item per screen and the user scrolls vertically to move to the next one; items matching the user's watchlist are boosted. The watchlist screen (Figure 2.9, right) lets the user add and remove tickers. The navigation bar with Discussions, News Reel, Personal page, Search and Notifications is persistent on all screens, and the user can switch between light and dark mode. Calls detected by AI always carry a visible "AI-inferred" label. All screens show clear loading, empty and error states and adapt to desktop and mobile widths.

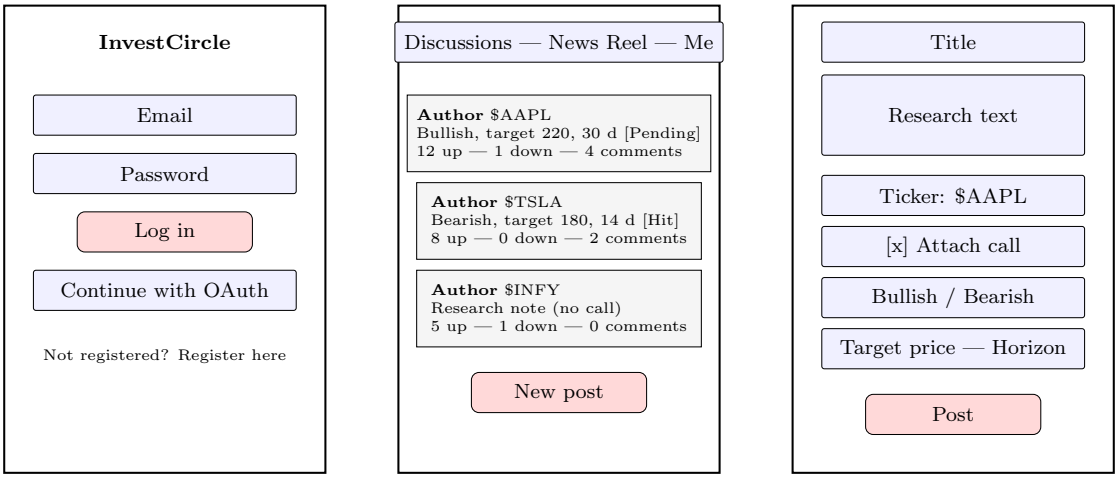


Figure 2.8: Login, Discussions feed and post composer (low-fidelity prototypes)

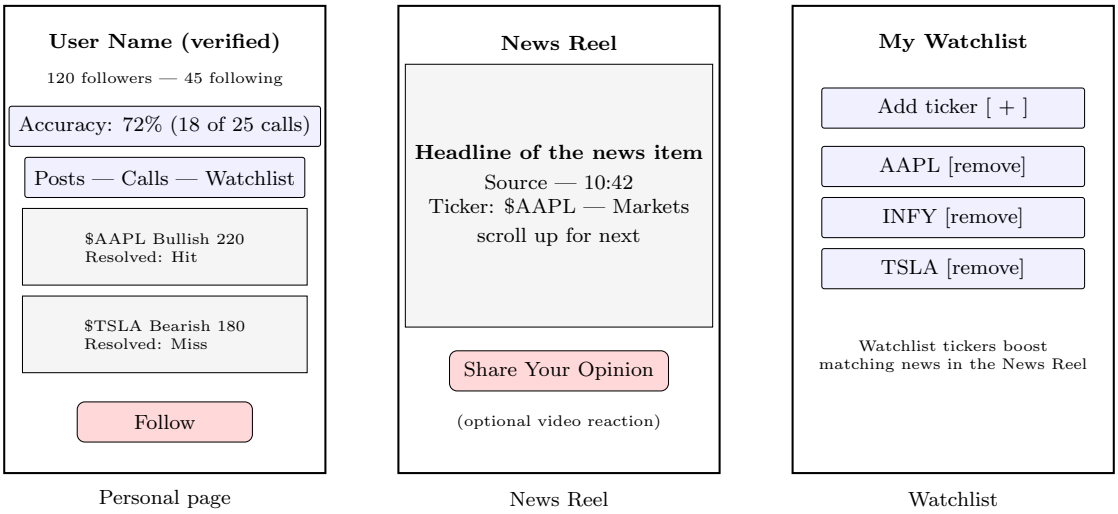


Figure 2.9: Personal page, News Reel and Watchlist (low-fidelity prototypes)

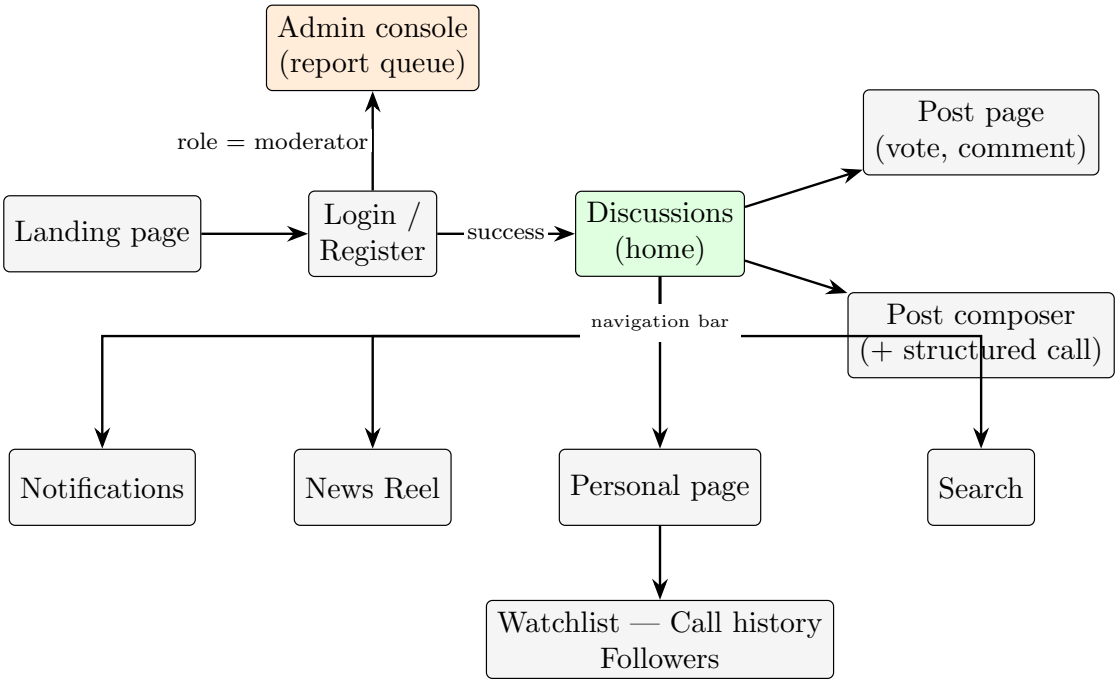


Figure 2.10: Screen navigation flow of the website

Hardware interfaces. Since the website has no designated hardware, it does not have any direct hardware interfaces. The connection between the client device and the server is managed by the underlying operating system and browser. A camera and microphone are used only by the optional video-reaction feature.

Software interfaces. The server communicates with the market-data API (Alpha Vantage, Finnhub or similar) through the provider-abstraction layer to obtain price history, quotes and market news; responses are cached in Redis. It communicates with PostgreSQL through an ORM, with Redis for cache and job queue, with an optional OAuth provider for federated login, and with the optional AI service for tagging and inferred-call detection. The AI service is never on the critical path.

Communications interfaces. All communication between the web client and the server uses HTTPS with JSON over a REST API. Authenticated requests carry a JWT in the Authorization header. Large lists are returned with page-based or cursor-based pagination. Email notifications, if enabled, use an SMTP or transactional-email service.

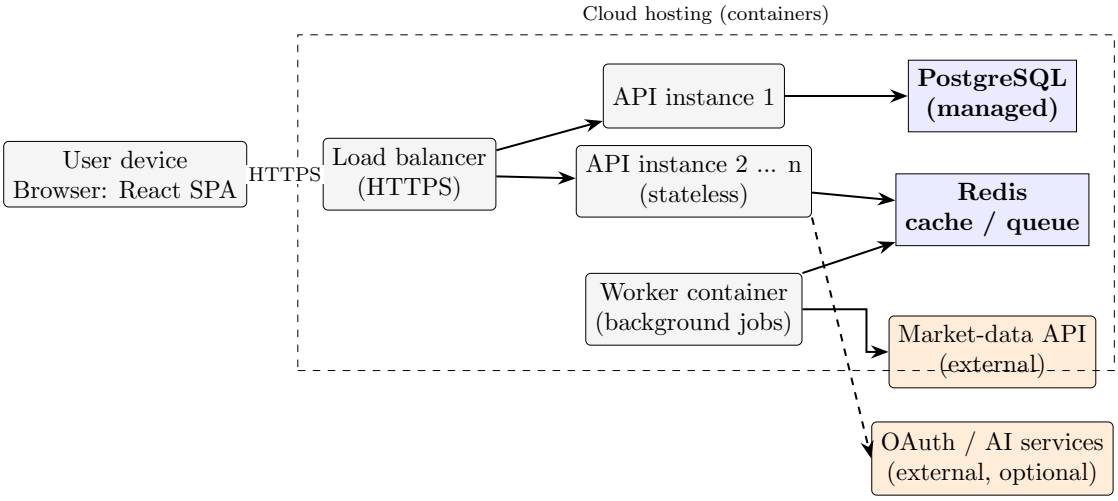


Figure 2.11: Deployment diagram

Design and implementation constraints

- Stack: React, Node.js REST backend, PostgreSQL, Redis, JWT authentication.
- Market data only through the external provider, subject to its rate limits and terms of use; the provider must be isolated behind a service layer. Since the provider limits requests, call resolution and news synchronisation may be delayed.
- Passwords hashed with bcrypt or Argon2; database access only through an ORM or parameterised queries.
- No trade execution or brokerage integration.
- Authentication, discussions, watchlists and explicit call tracking must stay fully functional if optional AI components are unavailable.
- The database is shared by all API instances; heavy load may queue requests and increase response time, which is reduced by indexing, caching and pagination.
- Four-person team, four months, short iterations with explicit module ownership; Git and CI/CD.

Assumptions and dependencies

- The provider supplies price history sufficient to evaluate calls, and its free-tier limits are adequate for project-scale use with caching. If the provider changes its terms or goes down, call resolution and the News Reel are affected, but discussions still work.
- Users register with a valid email address or an OAuth account, and use a browser that supports a current version of JavaScript.
- A call is evaluated using the price data of the exchange where the ticker is primarily listed.
- **Hit/Miss rule:** a bullish call is a Hit if the price reaches or exceeds the target at any point within the horizon; a bearish call is a Hit if it reaches or falls below the target (evaluated on daily high/low data); otherwise it is a Miss. This fixes the proposal's "checks whether the target price was reached" so that it can be tested unambiguously.

Apportioning of requirements

If the project is delayed, some requirements can be moved to later versions. The optional extensions (AI-inferred calls, video reactions, advanced recommendations, analytics and rankings) are developed in the third release and are the first to be dropped. If more time is lost, contributor statistics, sector/sentiment tagging and the personalisation of the News Reel can also be moved to a later version, because the core accountability workflow (posts, calls, resolution and track record) does not depend on them. See Section 3.1 for the release plan.

2.5 Design

Contributor activities have 7 steps - the core workflow of "InvestCircle":

1. A registered user creates a discussion post.
2. The user may attach a structured call (ticker, bullish/bearish, target price, time horizon).
3. The system stores the call with status Pending.
4. When the horizon expires, a background process obtains the relevant market price.
5. The system checks whether the target price was reached.
6. The call is marked Hit or Miss, and the resolution timestamp and data source are stored.
7. The result becomes part of the user's public track record on the Personal page.

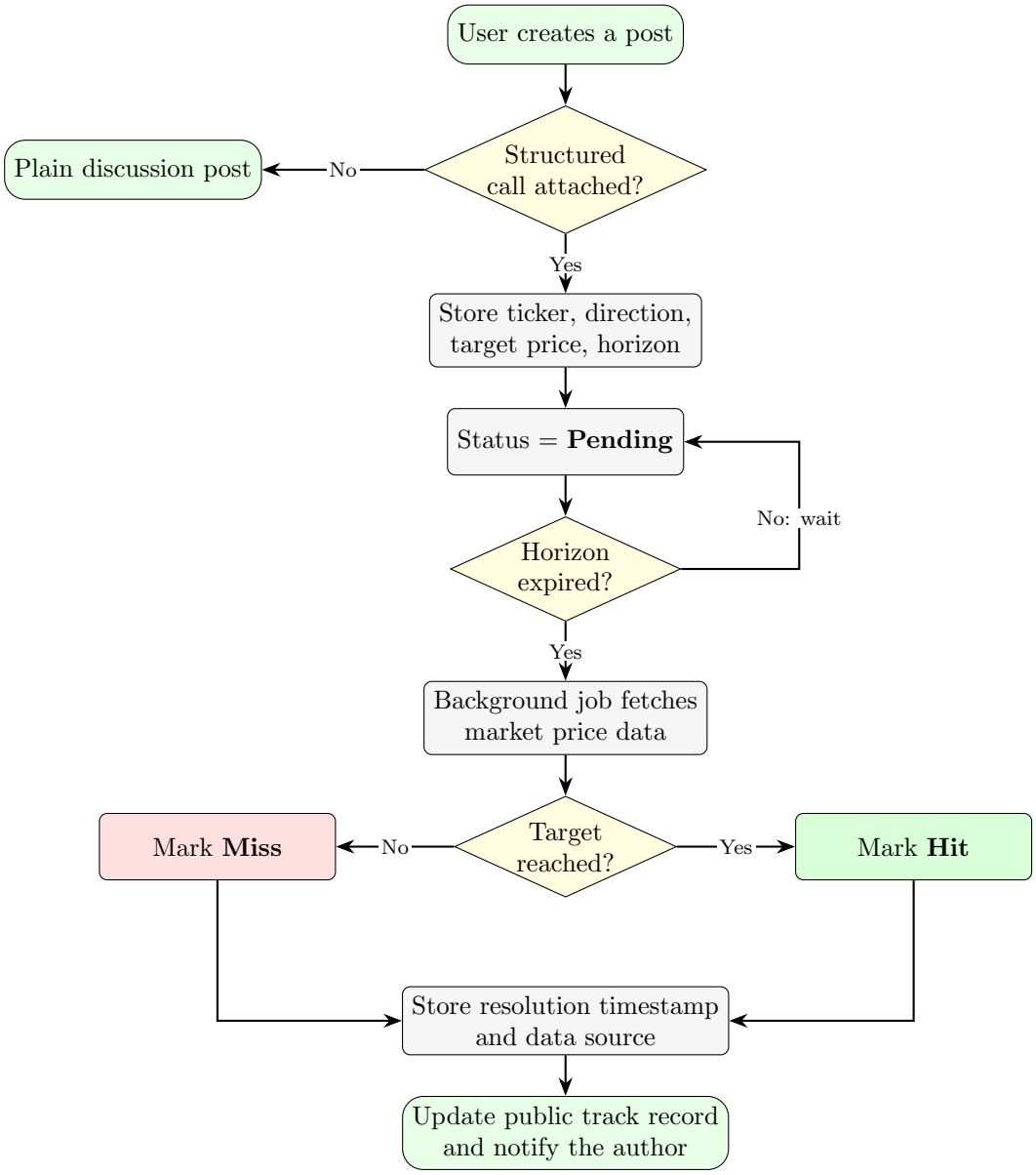


Figure 2.12: Stock-call resolution workflow

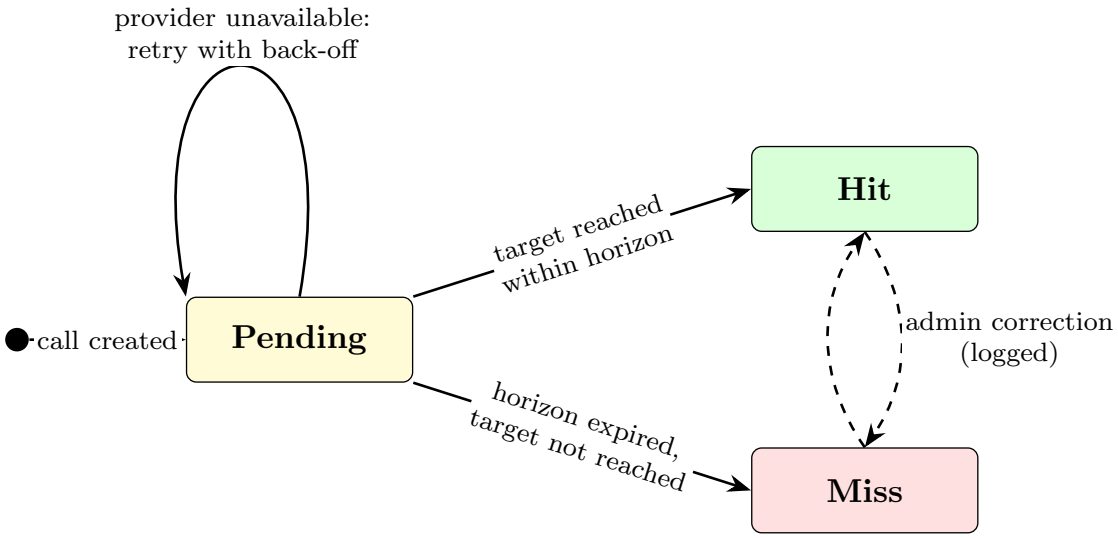


Figure 2.13: State diagram of a call

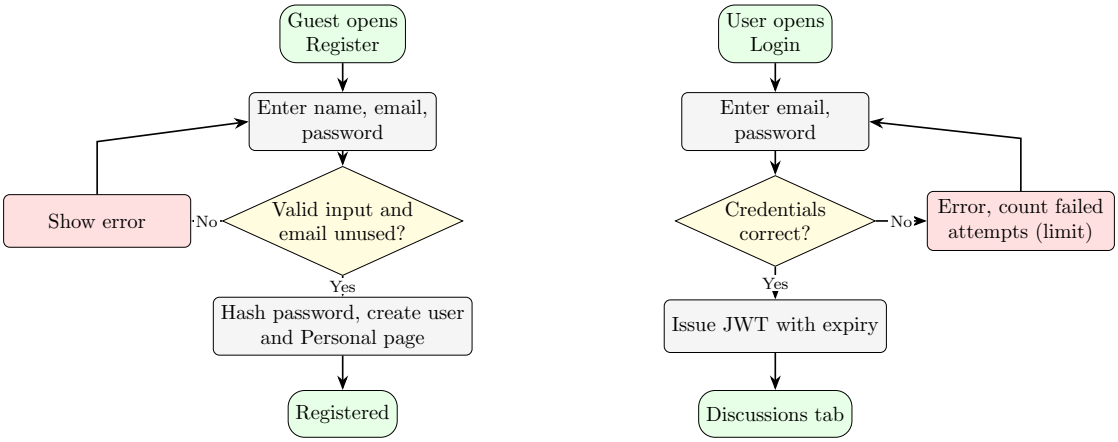


Figure 2.14: Registration and login flow

Reader activities have 3 steps - Register / log in, Browse (Discussions feed, News Reel, search), and Participate (vote, comment, reply, follow, maintain watchlists). Moderator / Admin activities are: review reported content, remove content, suspend accounts, and correct a resolved call (logged). Every user's Personal page contains identity information, biography, followers, following, own posts and the historical call results with an overall accuracy score.

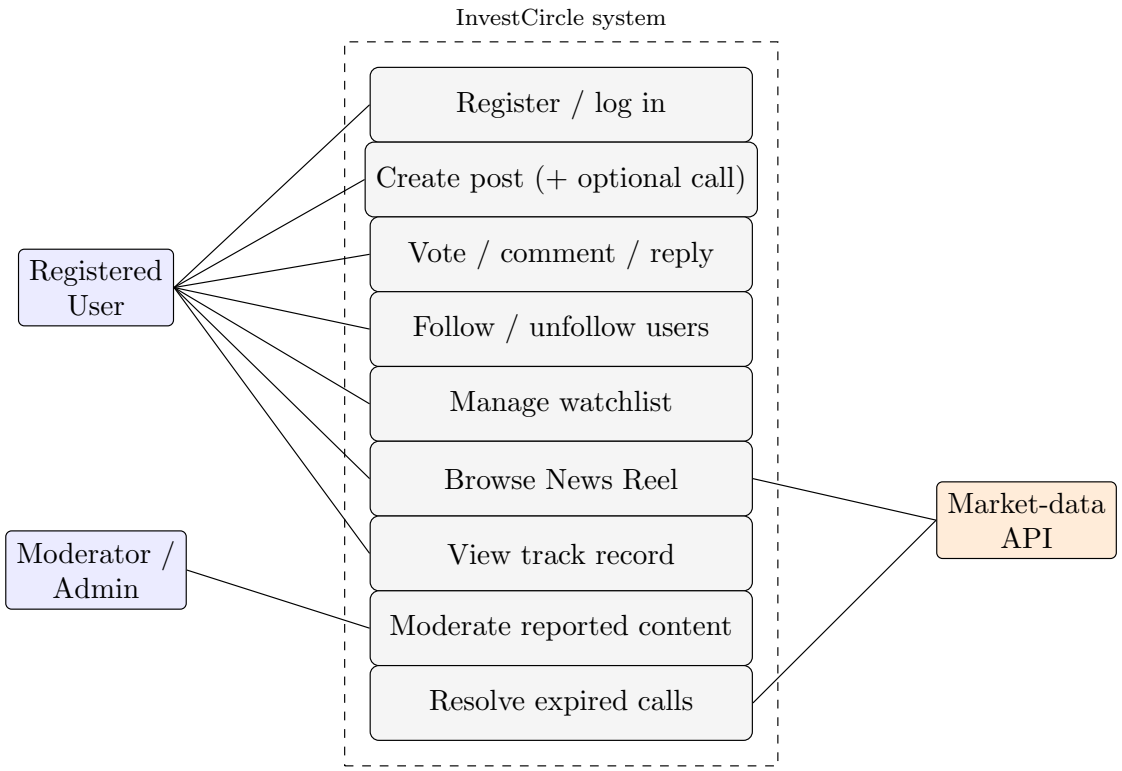


Figure 2.15: Use-case diagram (User, Moderator and external system activities)

UC-1 Publish a structured call. Actor: registered user, who must be logged in. The user opens the composer, writes the research text, enables "Attach call" and enters direction, target price and horizon. The system validates the input (positive target price, known ticker, future horizon), stores the post and call as Pending and shows it in the feed with a Pending badge. If validation fails an error is shown and nothing is stored.

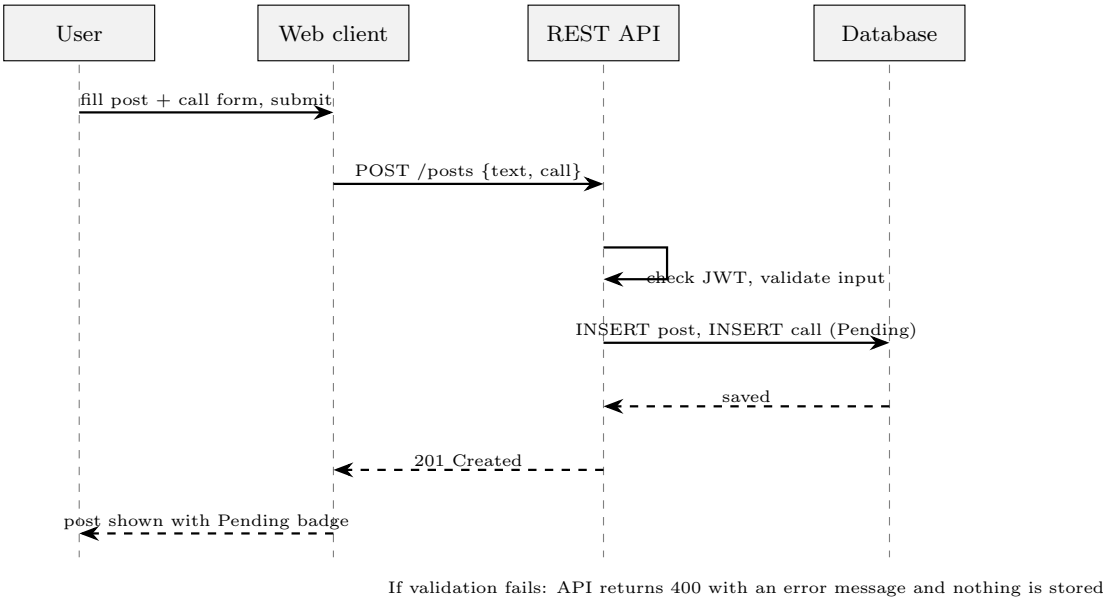


Figure 2.16: Sequence diagram of UC-1 (publish a structured call)

UC-2 Resolve an expired call. Actor: background worker and market-data API. The scheduler selects due Pending calls, fetches price data (cache first), applies the Hit/Miss rule, stores status, timestamp and data source, recomputes the author's accuracy and creates a notification. If the provider is unavailable, the call stays Pending and is retried with back-off.

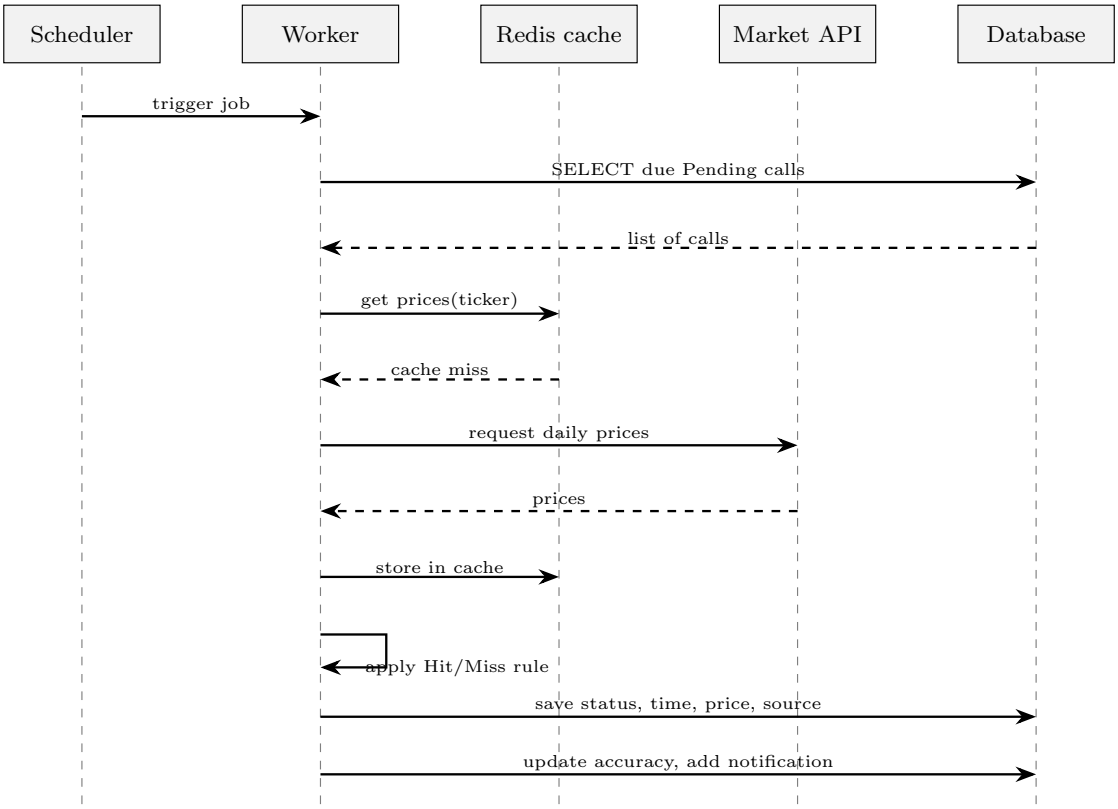


Figure 2.17: Sequence diagram of UC-2 (resolve an expired call)

UC-3 Moderate reported content. Actor: moderator. A user reports a post or comment; the moderator opens the report queue, reviews the content, and either dismisses the report or removes the content and, if needed, suspends the account. Every action is written to the moderation log.

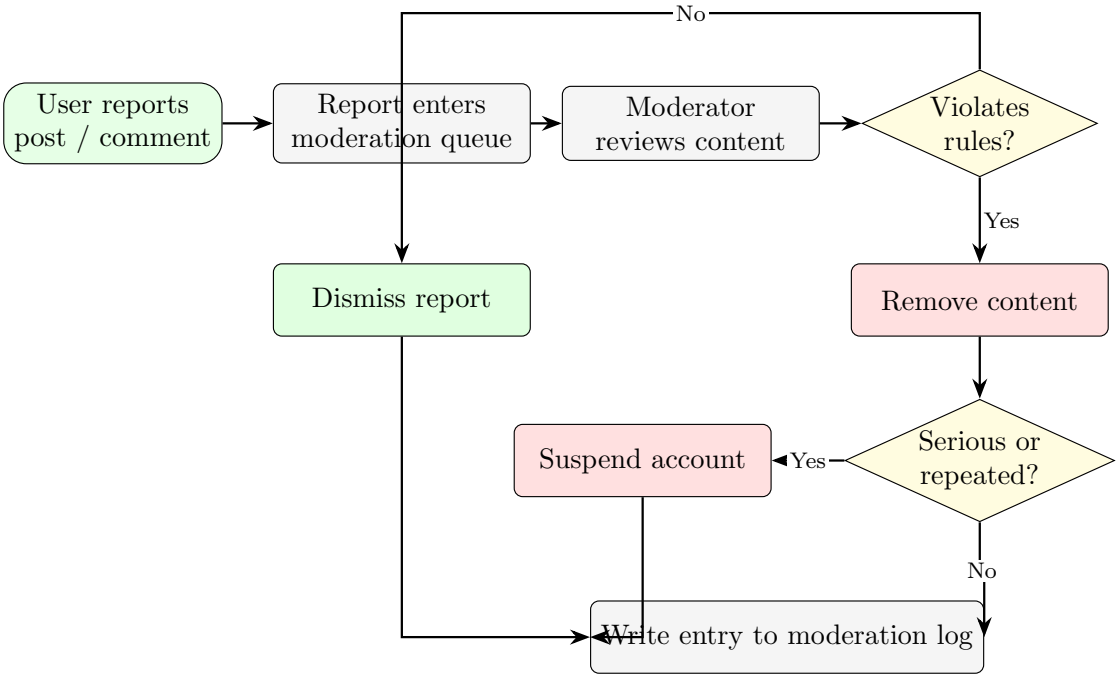


Figure 2.18: Moderation flow

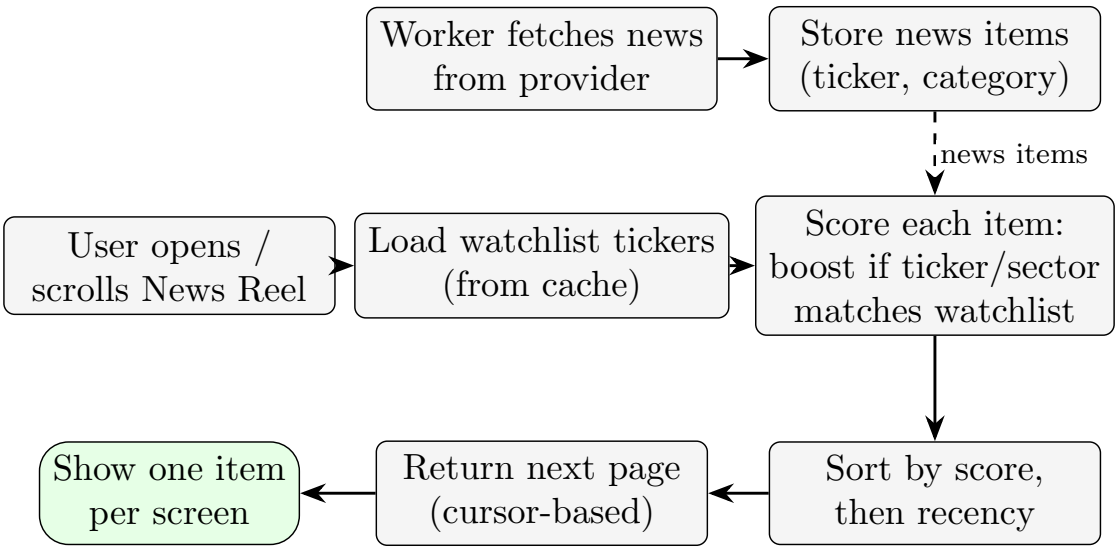


Figure 2.19: News Reel ranking flow

3 System Features

"InvestCircle" is a stock research and call-tracking web software. So the main art of this product is to publish structured calls and resolve them automatically against real market data. This chapter lists the features, explains how they were prioritised and released, and specifies the functional requirements in detail.

3.1 Description and Priority

"InvestCircle" has features that are main and also some are sub. The features with priority up to down -

1. User and Identity System (Must, MVP) : registration, authentication and profiles under real identity.
2. Discussion Posts and Voting (Must, MVP) : the Reddit-style Discussions tab.
3. Structured Stock Calls (Must, MVP) : this is the goal feature of this software, attached to posts and stored as Pending.
4. Follow / Unfollow and Basic Watchlists (Must, MVP).
5. Call Resolution and Track Record (Must/Should, next iteration) : the call data model is built in the MVP; automatic price-based resolution follows as soon as the core API is stable.
6. Comments and Threaded Replies, Personalised News Reel, Notifications, Search, Contributor Statistics, Basic Moderation (Should, subsequent iterations).
7. Sector/Sentiment Tagging, AI-Inferred Calls, Video Reactions, Advanced Recommendations (Could, optional extensions).

The first meaningful milestone is a working platform where a user can register, publish a structured call, follow another user, maintain a watchlist and eventually view the resolved outcome of the call.

Choice of prioritisation method

The requirements are prioritised with the MoSCoW method (Must, Should, Could). Must requirements form the foundation without which the platform cannot demonstrate its accountability idea; Should requirements make the product complete and attractive but are not vital for a first working version; Could requirements are optional extensions implemented only if development capacity remains. This method was chosen because it is simple, gives the whole team a shared vocabulary, and maps directly onto the releases below. Other methods, such as the hundred-dollar test, were considered, but with a four-person team and a fixed four-month schedule a finer ranking was not needed, and a single person could distort the result by spending all points on one requirement. A plain yes/no vote was rejected because its range is too narrow to separate requirements of similar importance. The dependencies between requirements (the DEP field) were also used to decide the order of the releases.

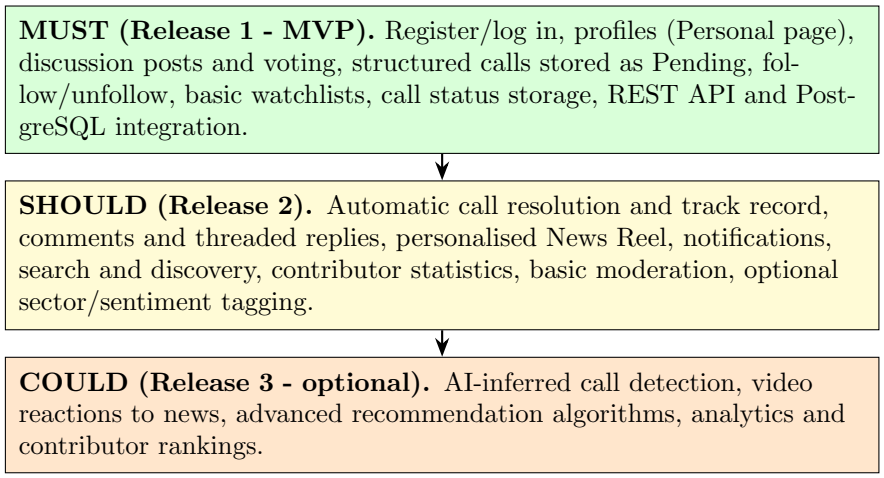


Figure 3.1: MoSCoW priority tiers of the features

Release plan

The requirements are divided into three releases based on priority and dependencies, so that each release works as a fully functional application.

Release	Requirements	Motivation
1 (MVP)	FR1, FR3, FR4, FR6–FR16, FR19–FR23, FR30, QR-series for security basics	Foundation of the application: users, discussions, structured calls stored as Pending, follow, watchlists, REST API and PostgreSQL integration. The first meaningful milestone is reached here.
2	FR17, FR18, FR24–FR29, FR31–FR34, FR35–FR42, FR2, FR5, FR43	Automatic call resolution and track record (the differentiating feature), comments, News Reel, notifications, search, statistics, moderation and optional tagging. Built after the core database and API are stable.
3	FR44, FR45 and advanced analytics	Optional extensions (AI-inferred calls, video reactions, recommendations, rankings): implemented only if capacity remains, and the first to be dropped if the project is delayed.

Table 3.1: Release plan

Quality requirements QR1 to QR6 are considered continuously during the whole process and can be closed only when all functions are implemented. The project is developed in short iterations, with each module tested independently before integration.

3.2 Functional Requirements

The ”InvestCircle” website is being build on React (Frontend), Node.js with a REST framework (Backend), PostgreSQL (Database), Redis (cache and jobs) and JWT authentication. This section includes the requirements that specify all the fundamental actions of the software system, grouped by user class. Priorities (Must/Should/Could) are given in the DESC field.

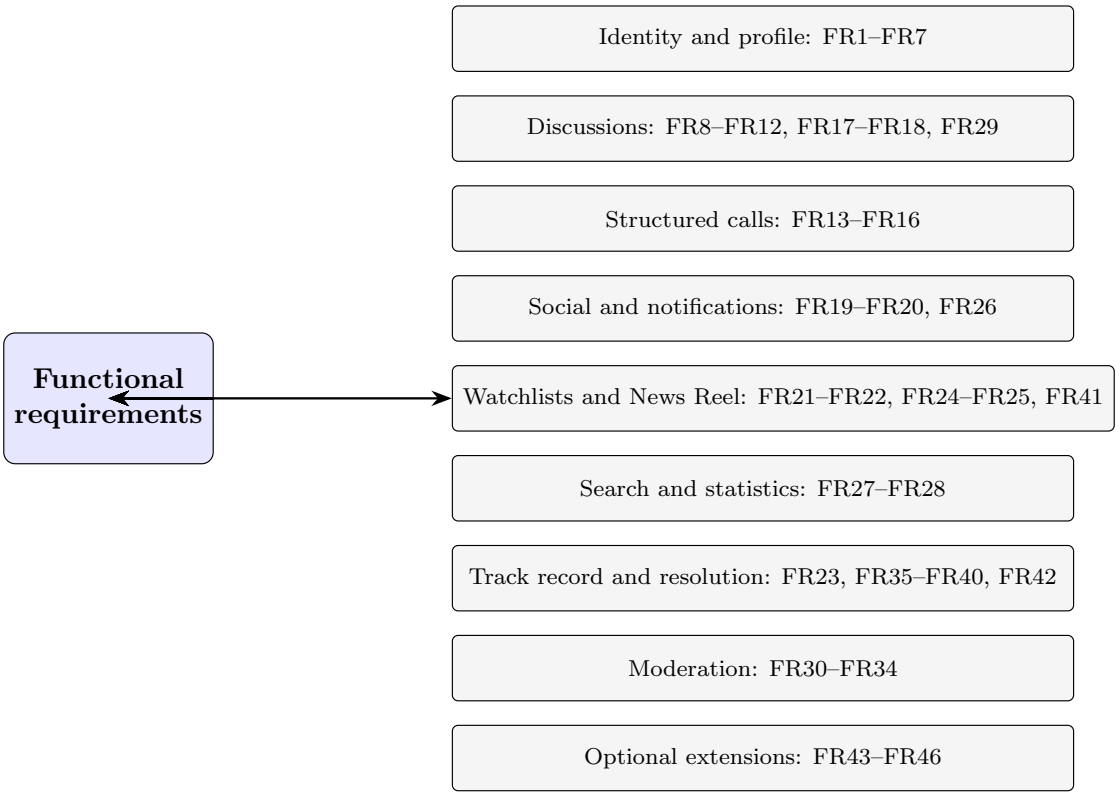


Figure 3.2: Overview of the functional requirement groups

User Class 1 - The User (Guest, Reader, Contributor)

ID: FR1
TITLE: Register account
DESC: Given that a guest is not registered, the guest should be able to register on the website by providing full name, email address and password. The system validates the email format and password strength and rejects an email that already exists. The Personal page is created on successful registration. (Must)
RAT: In order for a guest to become a registered user.
DEP: None

ID: FR2
TITLE: Register and log in with OAuth
DESC: A guest may register or log in using an OAuth provider instead of email and password. (Should)
RAT: In order to give a quicker and safer sign-up.
DEP: FR1

ID: FR3
TITLE: User log-in
DESC: Given that a user has registered, the user should be able to log in with email and password. On success the system issues a JWT access token with an expiry time; on failure the user is informed and no token is issued. Passwords are stored only as bcrypt/Argon2 hashes. (Must)
RAT: In order for a user to use the main functions of the website.
DEP: FR1

ID: FR4
TITLE: User log-out and session expiry
DESC: A logged-in user should be able to log out, which ends the session. Requests with a missing, invalid or expired token must be rejected and the user asked to log in again. (Must)
RAT: In order to protect accounts on shared devices.
DEP: FR3

ID: FR5
TITLE: Retrieve password
DESC: Given that a user has registered, the user should be able to reset the password through a link sent to the registered email address. (Should)
RAT: In order for a user to regain access to an account.
DEP: FR1

ID: FR6
TITLE: View Personal page
DESC: Every user has a persistent Personal page showing identity information, biography, follower and following counts, own posts and questions, watchlist and the history of calls. Other users can open the Personal page of a contributor. All content is displayed with the author's real identity, not an anonymous handle. (Must)
RAT: In order to show the identity and track record behind every post.
DEP: FR3

ID: FR7
TITLE: Edit profile
DESC: A user should be able to edit the biography and display information of his/her own profile. A user can edit only his/her own profile. (Must)
RAT: In order for a user to keep the profile accurate.
DEP: FR6

ID: FR8
TITLE: Create post
DESC: Given that a user is logged in, the user should be able to create a post containing a title, research text and an optional ticker. The system stores the post with the author, timestamp and zero votes. (Must)
RAT: In order for a user to share research, opinions and questions.
DEP: FR3

ID: FR9
TITLE: View Discussions feed
DESC: The Discussions tab shows posts as a feed. Each item shows the author, ticker, short text, vote and comment counts and, if present, the call badge with its status. The feed uses page-based or cursor-based pagination and never loads the complete history at once. (Must)
RAT: In order to browse discussions efficiently.
DEP: FR8

ID: FR10
TITLE: Filter feed by ticker
DESC: A user should be able to filter the Discussions feed by a ticker so that only posts about that stock are shown. (Should)
RAT: In order to find research about a specific stock.
DEP: FR9

ID: FR11
TITLE: Vote on post
DESC: A user should be able to upvote or downvote a post. A user has at most one vote per post and can change or retract it. (Must)
RAT: In order to surface useful discussions.
DEP: FR9

ID: FR12
TITLE: Edit and delete own post
DESC: A user should be able to edit or delete only his/her own posts. If a deleted post carries a call, the call remains in the author's track record. (Must)
RAT: In order for a user to correct mistakes without erasing accountability.
DEP: FR8

ID: FR13
TITLE: Attach structured call to post
DESC: When creating a post, a user may attach one structured call consisting of ticker, direction (bullish or bearish), target price and time horizon. A post carries at most one call.

- *Scenario - valid call:* Given the user is logged in and writing a post, when the user enables "Attach call" and enters a valid ticker, direction, positive target price and a future horizon, then the post and call are stored and the call status is Pending.
- *Scenario - invalid call:* Given the same situation, when the target price is not positive or the horizon is not in the future, then the user is informed and nothing is stored.

(Must)
RAT: This is the main feature of the system; it makes predictions identifiable and measurable.
DEP: FR8

ID: FR14
TITLE: Validate call input
DESC: The system validates that the ticker exists, that the target price is a positive number and that the horizon is a future date within an allowed maximum. If the system receives an invalid input the user is informed and prompted to insert an accepted input. (Must)
RAT: In order to store only evaluable calls.
DEP: FR13

ID: FR15
TITLE: Store call as Pending
DESC: On creation the call is stored with status Pending, the creation time and the reference price at creation. The core fields of a call (ticker, direction, target price, horizon) cannot be edited afterwards. Calls carry a flag distinguishing explicit calls from AI-inferred calls. (Must)
RAT: In order to preserve accountability and to allow later evaluation.
DEP: FR13, FR14

ID: FR16
TITLE: Show call status badge
DESC: Wherever a call is displayed (feed, post page, Personal page) its status is shown as a badge: Pending, Hit or Miss. (Must)
RAT: In order for readers to see outcomes at a glance.
DEP: FR15

ID: FR17
TITLE: Comment on post
DESC: A logged-in user should be able to write a comment on a post. (Should)
RAT: In order to discuss research with other users.
DEP: FR8

ID: FR18
TITLE: Reply to comment (threaded)
DESC: A user should be able to reply to a comment. Replies refer to their parent comment so that discussions form threads. (Should)
RAT: In order to keep discussions organised.
DEP: FR17

ID: FR19
TITLE: Follow and unfollow users
DESC: A user should be able to follow and unfollow another user. A user cannot follow himself/herself and cannot follow the same user twice. (Must)
RAT: In order to track contributors of interest.
DEP: FR6

ID: FR20
TITLE: View followers and following
DESC: Follower and following counts and lists are visible on the Personal page. The feed can be filtered to show posts of followed users. (Must)
RAT: In order to see a contributor's reach and to personalise the feed.
DEP: FR19

ID: FR21
TITLE: Create and manage watchlists
DESC: A user should be able to create a watchlist, give it a name and delete it. (Must)
RAT: In order for a user to organise stocks of interest.
DEP: FR3

ID: FR22
TITLE: Add and remove tickers in watchlist
DESC: A user should be able to add a ticker to and remove a ticker from a watchlist. A duplicate ticker in the same watchlist is rejected and the user informed. (Must)
RAT: In order to maintain the list of stocks the user follows.
DEP: FR21

ID: FR23
TITLE: View track record and accuracy
DESC: The Personal page shows the user's overall accuracy score and the list of resolved calls with outcome. The displayed call history must correspond exactly to the stored resolution records. Explicit and AI-inferred calls are shown and counted separately, never silently mixed. (Must)
RAT: This is the credibility signal of the platform.
DEP: FR6, FR38

ID: FR24
TITLE: View News Reel
DESC: The News Reel is a fast, vertically scrolling, reels-style feed of market news. Each item shows headline, source, publication timestamp, related ticker and market category. New items are loaded with cursor-based pagination as the user scrolls. (Should)
RAT: In order to replace unproductive scrolling with useful market information.
DEP: FR41

ID: FR25
TITLE: Personalise News Reel by watchlist
DESC: News items related to tickers or sectors in the user's watchlist are ranked higher using a simple deterministic boosting rule. No complex recommendation model is used in the first implementation. (Should)
RAT: In order to show the most relevant news first.
DEP: FR22, FR24

ID: FR26
TITLE: Receive notifications
DESC: The system generates a notification for a new follower, a comment or reply on the user's post and a resolved call. The user can list notifications and mark them as read. (Should)
RAT: In order to keep users informed without polling the site.
DEP: FR19, FR17, FR40

ID: FR27
TITLE: Search and discovery
DESC: A user should be able to search users, stocks (tickers) and discussions with a free-text query and open a result. (Should)
RAT: In order to find contributors and research quickly.
DEP: FR8

ID: FR28
TITLE: View contributor statistics
DESC: The system shows statistics for a contributor: number of calls, hit rate, accuracy per ticker (and per sector where available). (Should)
RAT: In order to evaluate contributors in more detail.
DEP: FR38, FR40

ID: FR29
TITLE: Report content
DESC: A user should be able to report a post or comment to the moderators. (Should)
RAT: In order to help keep the platform clean.
DEP: FR8

User Class 2 - Moderator / Administrator

ID: FR30
TITLE: Moderator log-in and permissions
DESC: A moderator or administrator logs in with a privileged account. Privileged operations are available only to users with the required role; all other users are refused. (Must)
RAT: In order to protect administrative functions.
DEP: FR3

ID: FR31
TITLE: Review reports and remove content
DESC:

- *Scenario - remove:* Given the moderator is logged in and a report exists, when the moderator removes the reported content, then the content is no longer visible and the action is logged.
- *Scenario - dismiss:* when the moderator dismisses the report, then the content stays and the report is closed.

(Should)
RAT: In order to enforce the terms of use.
DEP: FR29, FR30

ID: FR32
TITLE: Suspend and reactivate accounts
DESC: A moderator should be able to suspend an account that violates the rules and to reactivate it. A suspended user cannot log in. (Should)
RAT: In order to stop repeated abuse.
DEP: FR30

ID: FR33
TITLE: Correct a resolved call
DESC: Resolved calls are immutable. Only an administrator may correct a wrongly resolved call, for example after a data error; the correction is logged with reason, time and administrator. (Should)
RAT: In order to handle data errors without losing auditability.
DEP: FR38, FR30

ID: FR34
TITLE: View moderation log
DESC: An administrator should be able to view the log of all moderation actions and call corrections. (Should)
RAT: In order to audit privileged actions.
DEP: FR31, FR33

User Class 3 - System (background processing)

ID: FR35
TITLE: Select due calls
DESC: A background process periodically selects calls whose status is Pending and whose horizon has expired. (Must)
RAT: In order to start resolution automatically without manual work.
DEP: FR15

ID: FR36
TITLE: Fetch market data
DESC: For each due call the system obtains the relevant market-price data from the market-data provider through the provider-abstraction layer, using cached data first. (Must)
RAT: In order to evaluate calls against real prices.
DEP: FR35

ID: FR37
TITLE: Determine Hit or Miss
DESC: The system applies the Hit/Miss rule of Section 2.4 (assumption on the Hit/Miss rule) and sets the status to Hit or Miss.

- *Scenario - hit:* Given a bullish call with target 220 whose horizon has expired and the daily high reached 221 within the horizon, then the call is marked Hit.
- *Scenario - miss:* Given the same call where the highest price was 215, then the call is marked Miss.

(Must)
RAT: This is the core business logic of the platform.
DEP: FR36

ID: FR38
TITLE: Store resolution record
DESC: For every resolved call the system stores the status, resolution timestamp, resolution price and the data source used. (Must)
RAT: In order to make every result auditable.
DEP: FR37

ID: FR39
TITLE: Retry on market-data failure
DESC: If market data is unavailable or rate-limited, the call stays Pending and is retried with back-off. Persistent failure is logged and flagged for review. (Must)
RAT: In order to reach the reliability target despite provider downtime.
DEP: FR36

ID: FR40
TITLE: Update accuracy and notify
DESC: After a resolution the system updates the author's accuracy statistics (hits, misses, accuracy percentage) and generates a notification for the author. (Must)
RAT: In order to keep the track record up to date.
DEP: FR38

ID: FR41
TITLE: Synchronise market news
DESC: The system periodically fetches market news (headline, source, timestamp, ticker, category) from the provider and stores it as news items. (Should)
RAT: In order to feed the News Reel.
DEP: None

ID: FR42
TITLE: Separate explicit and AI-inferred calls
DESC: All statistics and displays treat explicit and AI-inferred calls separately and label AI-inferred calls as "AI-inferred". (Must)
RAT: In order never to mix unconfirmed calls into the confirmed track record.
DEP: FR15

Optional extensions

ID: FR43
TITLE: Sector and sentiment tagging
DESC: At post creation an optional AI component classifies the post into a sector and a sentiment (positive, neutral, negative). (Could)
RAT: In order to make posts easier to filter and analyse.
DEP: FR8

ID: FR44
TITLE: AI-inferred call detection
DESC: An optional AI component reads free-text posts and detects an implied opinion (ticker plus bullish/bearish stance) when no structured call was filled in. Such inferred calls are checked against real price data after a set time window like explicit calls and are always labelled "AI-inferred". (Could)
RAT: In order to evaluate opinions that are not entered as structured calls.
DEP: FR35–FR38, FR42

ID: FR45
TITLE: Video reactions to news
DESC: A user may record a short video reaction to a news item ("Share Your Opinion"). A news item shows up to three video reactions from other users, prioritised by verified status and follower count. (Could)
RAT: In order to add personal opinion to the News Reel.
DEP: FR24

ID: FR46
TITLE: Independence from AI services
DESC: Authentication, discussions, watchlists and explicit call tracking must remain fully functional when the AI components are unavailable. (Must)
RAT: In order to keep the AI features outside the critical path.
DEP: FR43, FR44

4 Other Nonfunctional Requirements

The requirements in this chapter specify the quality of the system: its performance, security, quality attributes and the business rules. Quality requirements use the fields ID, TAG, GIST, SCALE, METER, MUST, PLAN and WISH. The MUST levels are taken from the project proposal; the PLAN and WISH levels are suggested stretch targets.

4.1 Performance Requirements

"InvestCircle" will be used for peer-to-peer stock research with growing users, posts and market data. So for more interaction Node.js, React, PostgreSQL and Redis are used. The requirements in this section give measurable limits for the system performance.

ID: QR1
TAG: FeedResponseTime
GIST: The speed with which the discussion and news feeds load.
SCALE: The time from request until the first page of the feed is returned and displayed.
METER: Measurements obtained from 1000 feed requests on a seeded project-scale dataset during load testing.
MUST: About 1.5 seconds or less for at least 95% of the requests.
PLAN: 1.0 second or less for 95% of the requests.
WISH: 0.5 second or less for 95% of the requests.

ID: QR2
TAG: QueryPerformance
GIST: The response time of database queries as the stored data grows.
SCALE: The time of the most frequent feed, profile and search queries under increasing seeded data volume (indexes on user IDs, tickers, timestamps).
METER: Query timing measurements at increasing seeded data sizes during testing.
MUST: Response time stays acceptable (within the limit of QR1) as seeded data grows.
PLAN: No noticeable increase up to 10 times the initial seeded data.
WISH: No noticeable increase up to 100 times the initial seeded data.

ID: QR3
TITLE: Pagination of large lists
DESC: Large feeds, search results and profile histories shall use page-based or cursor-based retrieval, so that the complete history is never loaded at once.
RAT: In order to keep response time and memory use low.

DEP: None
PRIORITY: Must

ID: QR4
TITLE: Caching
DESC: Market data and frequently requested feed data shall be cached in Redis to reduce repeated provider requests.
RAT: In order to respect provider rate limits and to speed up responses.
DEP: None
PRIORITY: Must

ID: QR5
TITLE: Asynchronous processing
DESC: Market-data synchronisation, expired-call resolution, notification generation and news-feed updates shall run in background workers and shall not block user requests.
RAT: In order to keep the website responsive.
DEP: FR35–FR41
PRIORITY: Must

ID: QR6
TITLE: Scalability
DESC: The backend shall be stateless so that several instances can run at the same time; data providers are isolated behind a service layer; the system can be deployed with containerised services and managed PostgreSQL.
RAT: In order to support growth of users, discussions and market data.
DEP: None
PRIORITY: Should

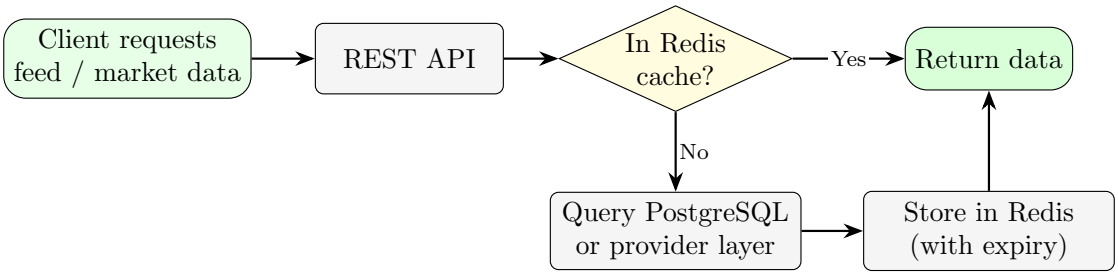


Figure 4.1: Caching flow for feeds and market data

4.2 Security Requirements

No one without registered users can enter the main features of the website. One particular user of a section only can perform his/her particular actions.

ID: QR7
TAG: PasswordProtection
GIST: Security of stored passwords.
SCALE: Passwords are stored only as a salted bcrypt or Argon2 hash and never in plain text or in logs.
METER: Inspection of the database and logs after 1000 registrations during testing.
MUST: 100% of stored passwords are hashes.
PLAN: -
WISH: -

ID: QR8
TAG: CommunicationSecurity
GIST: Security of the communication between client and server.
SCALE: All messages between the client and the server are encrypted with HTTPS, so that others cannot get passwords or tokens from them.
METER: Attempts to read credentials from captured traffic of 1000 log-in sessions during testing.
MUST: 100% of the communication messages are encrypted.
PLAN: -
WISH: -

ID: QR9
TAG: TokenAuthentication
GIST: Security of access to protected functions.
SCALE: A request with a missing, invalid or expired JWT must be rejected.
METER: 1000 requests with invalid, expired or missing tokens during testing.
MUST: 100% of such requests are rejected.
PLAN: -
WISH: -

ID: QR10
TAG: OwnershipAuthorisation
GIST: Security of user content.
SCALE: A user can edit or delete only his/her own resources; privileged operations need the moderator/admin role.
METER: 1000 attempts to modify another user's resource or to call a privileged operation without the role.
MUST: 100% of the attempts are refused.
PLAN: -
WISH: -

ID: QR11
TAG: LoginAttemptLimit
GIST: Protection of accounts against guessing of passwords.
SCALE: The number of failed log-in attempts allowed for an account and IP address within a time period (rate limiting).
METER: 1000 attempts to log in during the lock period after the limit has been reached.
MUST: After 5 failed attempts the log-in function is disabled for 15 minutes (proposed values).
PLAN: -
WISH: -

ID: QR12
TAG: InputValidationAndInjection
GIST: Security against malicious input.
SCALE: All API inputs are validated on the server; database access uses an ORM or parameterised queries; output is escaped.
METER: Security tests with 1000 injection and script payloads during testing.
MUST: No successful SQL injection or stored script execution.
PLAN: -
WISH: -

ID: QR13
TAG: DuplicateAccountSecurity
GIST: Security of creating accounts.
SCALE: If a user wants to register with an email address that is already used, the user is informed and no second account is created.
METER: Measurements obtained from 1000 registration attempts during testing.
MUST: 100% of the time.
PLAN: -
WISH: -

ID: QR14
TITLE: Rate limiting
DESC: Rate limiting is applied to log-in, posting and reporting operations.
RAT: In order to prevent spam and abuse.
DEP: None
PRIORITY: Should

ID: QR15
TITLE: Secret management
DESC: API keys and secrets are kept in environment configuration and never in source control.
RAT: In order to avoid leaking provider keys.
DEP: None
PRIORITY: Must

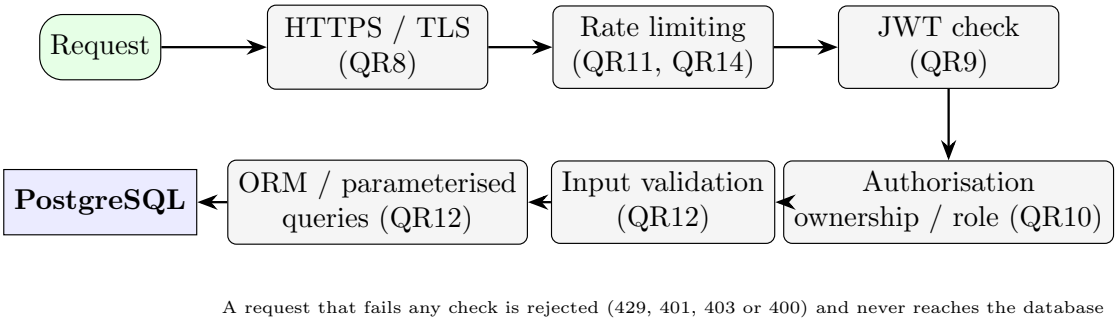


Figure 4.2: Layers of request security

4.3 Software Quality Attributes

In the development phase also testing and review of the team is been continued, so that the quality of the software is been maintained and all the requirements are been fulfilled. The requirements in this section specify the required reliability, availability, usability, maintainability and portability of the software system.

Reliability

ID: QR16
TAG: CallResolutionReliability
GIST: The reliability of the automatic call resolution.
SCALE: The share of due calls that are resolved to Hit or Miss without any manual intervention.
METER: Measurements obtained from a seeded set of at least 1000 due calls, including simulated provider failures, during testing.
MUST: At least 95% of the due calls.
PLAN: At least 98% of the due calls.
WISH: 100% of the due calls.

ID: QR17
TAG: ResultConsistency
GIST: The correctness of the displayed history.
SCALE: The share of displayed call histories that correspond exactly to the stored resolution records.
METER: Comparison of the displayed Personal pages with the database for 1000 contributors during testing.
MUST: 100% of the histories.

PLAN: -
WISH: -

ID: QR18
TAG: GracefulDegradation
GIST: The fault tolerance of the system.
SCALE: If the market-data provider or an AI service is unavailable, the core functions (authentication, discussions, watchlists, explicit call tracking) must stay available and the user is informed where a function is limited.
METER: Tests with the provider and the AI services switched off.
MUST: 100% of the core functions stay available without the AI services.
PLAN: -
WISH: -

ID: QR19
TITLE: Auditability
DESC: The resolution timestamp, resolution price and data source are stored for every resolved call; administrator corrections are logged.
RAT: In order to be able to trace and defend every result.
DEP: FR38, FR33
PRIORITY: Must

Availability

ID: QR20
TAG: SystemAvailability
GIST: The availability of the website when it is used.
SCALE: The average system availability (not considering failure of the user's own network).
METER: Measurements obtained from 1000 hours of operation of the test deployment.
MUST: More than 98% of the time.
PLAN: More than 99% of the time.
WISH: 100% of the time.

Usability

ID: QR21
TAG: EaseOfFirstUse
GIST: The ease with which a new user can start.
SCALE: The time for a new user to register and publish a first post with a structured call without help.
METER: Observation of at least 5 test users during testing.
MUST: Less than 5 minutes.
PLAN: Less than 3 minutes.
WISH: Less than 2 minutes.

ID: QR22
TITLE: Prominent main functions
DESC: The Discussions tab, News Reel and Personal page are reachable from a persistent navigation bar, and opening a post or a call takes only one click.
RAT: In order for a user to find the main functions easily.
DEP: None
PRIORITY: Must

ID: QR23
TITLE: Responsive interface
DESC: The interface adapts to desktop and mobile screen sizes and offers a light and a dark mode; forms show clear inline validation messages; text and controls have sufficient contrast and can be used with the keyboard.
RAT: In order to be usable on any device.
DEP: None
PRIORITY: Must

Maintainability and testability

ID: QR24
TITLE: Modularity
DESC: The code is modular with a layered architecture and explicit module ownership, and favors the implementation of new functions. The market-data provider can be replaced without changes to the business logic.
RAT: In order for future functions to be implemented easily.
DEP: None
PRIORITY: Must

ID: QR25
TAG: TestCoverage
GIST: The amount of automated testing.
SCALE: The share of backend and business-logic code covered by automated tests (unit tests of call resolution and feed ranking, integration tests of APIs and database operations).
METER: Coverage report generated by the CI pipeline.
MUST: At least 70% coverage.
PLAN: At least 80% coverage.
WISH: At least 90% coverage.

ID: QR26
TITLE: Continuous integration
DESC: The project is kept in Git and a CI pipeline runs the automated tests on every push or pull request. Frontend tests of critical user workflows and load tests of the feed and profile endpoints are part of the test plan.
RAT: In order to find faults early.
DEP: None
PRIORITY: Must

Portability

ID: QR27
TITLE: Browser and platform portability
DESC: The website works in current versions of Chrome, Firefox, Edge and Safari on Windows, macOS, Linux, Android and iOS, and the backend runs in containers on common cloud hosting.
RAT: In order to reach as many users as possible.
DEP: None
PRIORITY: Must

Acceptance criteria

Metric	Target
Call-resolution reliability	$\geq 95\%$ of due calls resolved automatically
Feed performance	Feeds load in ≈ 1.5 s under seeded workload
Test coverage	$\geq 70\%$ backend and business-logic coverage
Functional completeness	All Must requirements demonstrated end-to-end
Graceful degradation	Core functionality works with AI services disabled
Data consistency	Displayed call history matches stored resolution records

Table 4.1: Acceptance criteria

4.4 Business Rules

"InvestCircle" is for peer-to-peer stock research and for tracking the outcome of structured calls.

- **BR1:** Discussion is identity-based; anonymous posting is not permitted.
- **BR2:** A post carries at most one structured call; its core fields cannot be changed after creation.
- **BR3:** A call's result is decided only by market data, never by the author or by votes.
- **BR4:** AI-inferred calls are always labelled and counted separately from confirmed calls.
- **BR5:** Credibility is expressed through historical accuracy, not followers or votes.
- **BR6:** The platform offers research and discussion only, with no trade execution.
- **BR7:** A user can follow another user at most once and cannot follow himself/herself.
- **BR8:** The platform displays a disclaimer that content is community opinion, not investment advice, and that past call accuracy does not guarantee future results.
- **BR9:** Market data and news are used according to the provider's terms of service and attribution requirements.
- **BR10:** Only the data necessary for the service is collected from users, and the terms of use prohibit market manipulation, spam and abusive content.

5 Other Requirements

"InvestCircle" needs maintenance as it is a long process software. It will need re-factoring and further the requirements can be changed as the field is changing frequently. Other requirements are:

- **Documentation:** API documentation (OpenAPI), database schema documentation and a short user guide.
- **Deployment:** containerised deployment with environment-based configuration for development, testing and production; errors, failed resolutions and moderation actions are logged. English only in the first version; prices are shown in the currency reported by the provider.
- **Development process:** short iterations with explicit module ownership and monthly milestones for the four-person team; each module is tested independently before integration; external market-data integration is introduced after the core database and API workflow is stable.
- **Data retention:** calls and resolution records are kept permanently to preserve the public track record; resolved calls are immutable except for logged administrator corrections.
- **Open issues to confirm:** final market-data provider and rate-limit tier; the Hit/Miss rule; maximum horizon and supported exchanges; account-deletion policy versus permanent track records; which optional extensions will be attempted.

Risks and mitigations.

Risk	Impact	Mitigation
External API dependency	Rate limits or downtime delay news and call resolution	Caching, scheduled sync, provider abstraction, retry with back-off
Incorrect call resolution	Wrong Hit/Miss shown publicly	Store timestamp and data source; deterministic rule; unit tests
Scope creep	AI, video, recommendations delay the core	Keep optional features isolated from the MVP
Security risks	Unauthorised access or data leakage	Hashing, token auth, authorisation, validation, rate limiting
Performance degradation	Slow feeds and searches with more data	Pagination, indexing and caching from the start
Uneven team contribution	Delays and quality gaps	Explicit module ownership and monthly milestones

Table 5.1: Risk register

References.

1. InvestCircle: A Peer-to-Peer Stock Research and Community Platform - Project Proposal, UCS503, Thapar Institute of Engineering and Technology.

2. IEEE Software Engineering Standards Committee, "IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications", October 20, 1998.

3. Davis M. A., "Just Enough Requirements Management: Where Software Development Meets Marketing", Dorset House Publishing, 2005.

4. Alpha Vantage / Finnhub API documentation.